基于强化学习的PPTAgent代码讲解

PPTAgent中有两个地方调用了大模型进行内容生成，分别是**大纲生成**和**内容生成**。本代码就是用GSPO强化学习算法对这两个部分使用到的大模型（即代码中的Qwen2.5-0.5B-Instruct，在面试准备文档中改为Qwen2.5-7B-Instruct）做了训练以期带来效果上的优化。

运行代码

1. 安装依赖

```
代码块
cd backend && pip install -r requirements.txt
```

2. 配置 `.env` 文件，默认使用的是Deepseek

```
代码块

ART_MODEL=Qwen/Qwen2.5-0.5B-Instruct
MAX_SEQ_LEN=4096

# DeepSeek API 配置（用于奖励函数评估）
DEEPSEEK_API_KEY=your-deepseek-api-key
DEEPSEEK_BASE_URL=https://api.deepseek.com/v1/chat/completions
DEEPSEEK_MODEL=deepseek-chat
USE_DEEPSEEK_JUDGE=true

OUTLINE_NAME=outline01
OUTLINE_PROJECT=outline-training

CONTENT_NAME=content01
CONTENT_PROJECT=content-training
```

3. 训练大纲生成模型，训练过程会：

- 加载基础模型（默认 Qwen2.5-0.5B-Instruct）
- 使用 `topic.json` 中的主题进行训练
- 通过规则奖励函数 + DeepSeek API 评估进行强化学习
- 模型保存到 `output/{OUTLINE_NAME}-{timestamp}/`

```
代码块
cd outline && python train_trl.py
```

4. 训练大纲生成模型， 将 `TRAINED_MODEL_PATH` 替换为你训练好的模型，生成 `outline.jsonl`

```
代码块
python model_test.py
```

5. 训练内容生成模型，训练过程会：

- 加载 `outline/outline.jsonl` 中的大纲作为训练数据
- 训练模型根据完整大纲生成详细内容
- 模型保存到 `output/{CONTENT_NAME}-{timestamp}/`

代码块

```
cd ../content && python train_trl.py
```

6. 测试内容生成, 将 `TRAINED_MODEL_PATH` 替换为你训练好的内容生成模型，生成 `content.jsonl`

代码块

```
python model_test.py
```

核心代码解读

两个强化学习训练的脚本都采用了GSPO算法，这里以生成大纲的代码 `backend/outline/train_trl.py` 为例，介绍训练过程。

1. 训练配置信息

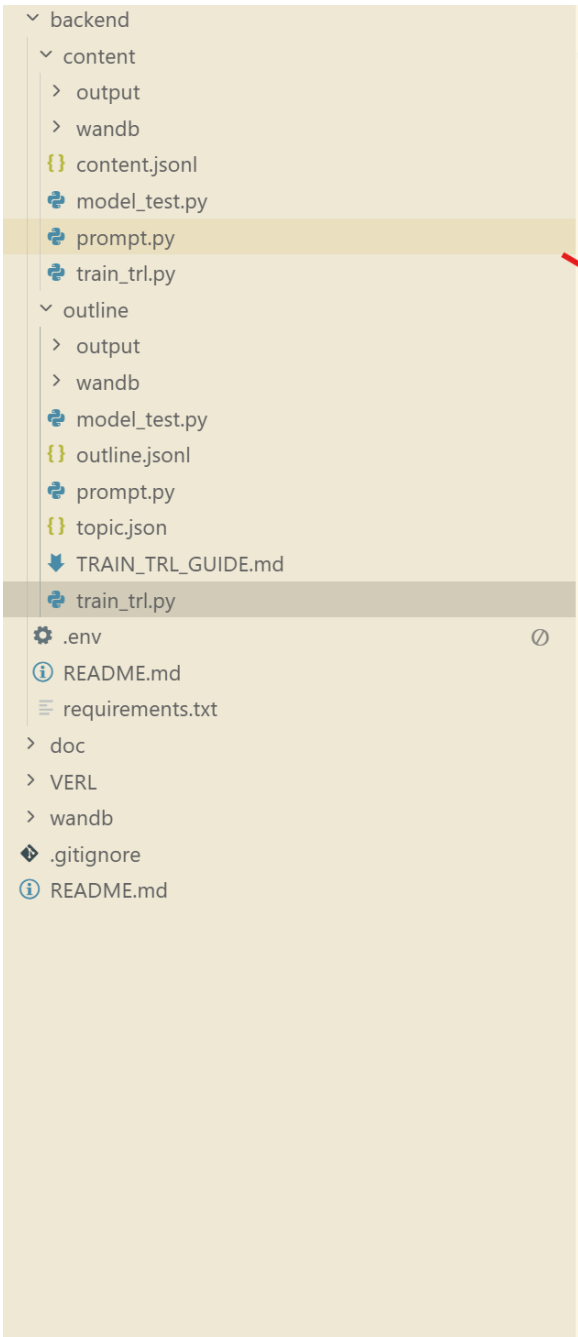
配置项	环境变量	默认值	说明
项目名称	OUTLINE_NAME	outline01	训练实验名称
基础模型	ART_MODEL	Qwen/Qwen2.5-0.5B-Instruct	HuggingFace 模型
项目名	OUTLINE_PROJECT	outline-training	WandB 项目名
序列长度	MAX_SEQ_LEN	4096	最大序列长度
启用API评估	USE_DEEPSEEK_JUDGE	TRUE	是否启用

2. 模型加载

代码块

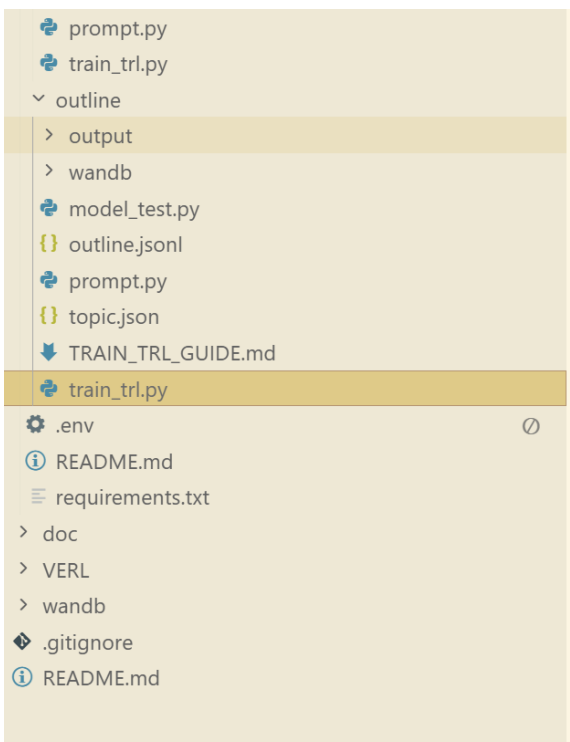
```
def load_model_and_tokenizer():
    """加载模型和tokenizer"""global tokenizer, model

    # 加载 Tokenizer
    tokenizer = AutoTokenizer.from_pretrained(
        MODEL_NAME,
        trust_remote_code=True,
        cache_dir=MODEL_CACHE_DIR
    )
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token  # 设置padding token
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_NAME,
        trust_remote_code=True,
        device_map="auto",          # 自动分配设备
        cache_dir=MODEL_CACHE_DIR,
        torch_dtype=torch.bfloat16  # 使用 bf16 节省显存
    )

    return model, tokenizer
```

3. 规则奖励，主要是格式奖励，要求模型生成的大纲能被正确的解析，并且要求大纲中包含白名单词语，不包含黑名单词语，可以根据自己的需求进行修改，




```python
def load_model_and_tokenizer():
        print(f"[GPU] 模型设备: {next(model.parameters()).device}")

    return model, tokenizer


# ============================================================
# 奖励函数 1: 规则奖励 (结构 + 格式)
# ============================================================
def rule_based_reward(completions: List[str], **kwargs) -> List[float]:
    """
    【奖励函数 1】基于规则的大纲评分 (归一化到 [0, 1])

    评估标准 (与 ROLLOUT_SYSTEM_PROMPT 对齐):
    - 结构: # 1个, ## 5个, 每个##下3-4个###, 每个###下3-5个要点
    - 格式: 要点以动词开头、≤18字、不以句号结尾
    - 禁止: 问句标题、禁止段落 (引言/结语/总结/目录/参考)
    """
    rewards = []
    for md in completions:
        try:
            score = 0.0

            # 1. 一级标题检查 (0.1分)
            h1 = re.findall(r"(?m)^# [^\n]+$", md)
            if len(h1) == 1:
                score += 0.1

            # 2. 二级标题检查 (0.2分)
            h2_matches = list[Match[str]](re.finditer(r"(?m)^## [^\n]+$", md))
            if len(h2_matches) == 5:
                score += 0.2
            elif len(h2_matches) > 0:
                score += 0.1 * min(len(h2_matches), 5) / 5

            # 3. 三级标题检查 (0.2分)
            h3_count = len(re.findall(r"(?m)^### [^\n]+$", md))
```

4. LLM-as-Judge奖励，引入大模型进行判断，从内容质量、结构完整、格式规范3个角度进行评价，然后归一化到 $[0,1]$ 区间内。




```python
def deepseek_judge_reward(completions: List[str], **kwargs) -> List[float]:
    """
    【奖励函数 2】DeepSeek API 评估奖励 (归一化到 [0, 1])

    调用 DeepSeek API 对大纲进行综合评估，重点关注内容质量。
    """
    if not USE_DEEPSEEK_JUDGE or not DEEPSEEK_API_KEY:
        # 未启用或无API Key，返回中间值
        return [0.5] * len(completions)

    rewards = []
    for completion in completions:
        score = _call_deepseek_judge(completion)
        rewards.append(score)
    return rewards


# ============================================================
```

5. 数据集准备，将用户提问加入到prompt中，将消息转换为Qwen2.5特定的格式。

6. GSPO 训练配置，GSPO与GRPO的核心区别就在于重要性采样，GRPO是token-level的，GSPO是sequence-level的，在代码中体现在 `importance_sampling_level` 参数。核心参数为：

参数	值	说明
num_train_epochs	1	训练轮数，完整遍历数据集的次数
per_device_train_batch_size	1	每个 GPU 的批次大小
gradient_accumulation_steps	4	梯度累积步数，等效 batch_size = 1×4 = 4
learning_rate	1.00E-06	学习率，GSPO 推荐使用较小值以保证训练稳定
weight_decay	0.1	权重衰减系数，用于 L2 正则化防止过拟合
warmup_ratio	0.1	预热比例，前 10% 的步数逐渐增加学习率
lr_scheduler_type	"cosine"	学习率调度策略，使用余弦退火
logging_steps	1	每隔多少步记录一次日志
save_steps	50	每隔多少步保存一次模型检查点
max_completion_length	MAX_SEQ_LEN // 2	模型生成内容的最大 token 数（2048）
num_generations	4	组大小，每个 prompt 生成的候选样本数
temperature	0.7	采样温度，控制生成多样性（越高越随机）
seed	42	随机种子，保证实验可复现
bf16	TRUE	使用 BFloat16 精度训练，节省显存
report_to	"wandb"	训练指标上报平台（Weights & Biases）
remove_unused_columns	FALSE	不移除数据集中未使用的列
num_iterations	1	每个 batch 的迭代次数
beta	0.04	KL 散度惩罚系数，防止模型偏离原始分布过远
importance_sampling_level	"sequence"	GSPO 核心参数：使用序列级重要性采样（而非 token 级）
mask_truncated_completions	FALSE	不对被截断的生成内容进行 mask
loss_type	"dr_grpo"	使用 Dr.GRPO 损失函数

代码块

```python
gspo_config = GRPOConfig(
        output_dir=output_dir,
        num_train_epochs=1,
        per_device_train_batch_size=1,
        gradient_accumulation_steps=4,
        learning_rate=1e-6,  # GSPO 推荐使用更小的学习率
        weight_decay=0.1,
        warmup_ratio=0.1,
        lr_scheduler_type="cosine",
        logging_steps=1,
        save_steps=50,
        max_completion_length=MAX_SEQ_LEN // 2,
        num_generations=4,  # 每个 prompt 生成4个样本进行比较
        temperature=0.7,
        seed=42,
        bf16=True,
        report_to="wandb",
        remove_unused_columns=False,
        num_iterations=1,
        beta=0.04,  # GSPO 推荐的 KL 惩罚系数
        # GSPO 核心参数：序列级别的重要性采样（区别于 GRPO 的 token 级别）
        importance_sampling_level="sequence",
        mask_truncated_completions = False,
        loss_type = "dr_grpo",
    )
```

7. 进行GSPO训练

代码块

```python
reward_funcs = [rule_based_reward]
if USE_DEEPSEEK_JUDGE and DEEPSEEK_API_KEY:
    reward_funcs.append(deepseek_judge_reward)
trainer = GRPOTrainer(
    model=model,                    # 要训练的模型
    args=gspo_config,               # GSPO 配置
    train_dataset=train_dataset,    # 训练数据集
    processing_class=tokenizer,     # 用于处理文本
    reward_funcs=reward_funcs,      # 奖励函数列表
)
```

8. 模型推理代码见 `backend/outline/model_test.py` ，加载训练好的模型，使用训练时相同的prompt模板进行推理。

backend > outline > model_test.py > NAME

- backend
  - content
    - output
    - wandb
    - content.jsonl
    - model_test.py
    - prompt.py
    - train_trl.py
  - outline
    - output
    - wandb
    - model_test.py
    - outline.jsonl
    - prompt.py
    - topic.json
    - train_trl.py
  - .env
  - README.md
  - requirements.txt
- doc
- VERL
- wandb
- .gitignore
- README.md

```python


def generate_outline(model, tokenizer, topic: str) -> str:
    """根据主题生成大纲"""
    # 构建消息
    messages = [
        {"role": "system", "content": prompt.ROLLOUT_SYSTEM_PROMPT},
        {"role": "user", "content": prompt.ROLLOUT_USER_PROMPT.format(topic=topic)}
    ]

    # 应用聊天模板
    if hasattr(tokenizer, 'apply_chat_template'):
        input_text = tokenizer.apply_chat_template(
            messages,
            tokenize=False,
            add_generation_prompt=True
        )
    else:
        input_text = f"System: {prompt.ROLLOUT_SYSTEM_PROMPT}\n\nUser: {prompt.ROLLOUT_USER_PROI

    # tokenize
    inputs = tokenizer(input_text, return_tensors="pt").to(model.device)

    # 生成
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=2048,
            temperature=0.7,
            do_sample=True,
            top_p=0.9,
            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )

    # 解码（只取生成的部分）
    generated_ids = outputs[0][inputs['input_ids'].shape[1]:]
```